A* Island Simulator: Empirical Analysis of Game Mechanics

Data Analysis Report

March 2026

Abstract

We analyze 45 replay datasets (9 rounds × 5 seeds, 51 frames each) to reverse-engineer the simulation mechanics of A* Island. We isolate the effects of food production, winter, settlement spawning, and raiding by examining step-to-step transitions. All findings are supported by reproducible scripts in `data-analysis/`.

1 Food Production and Winter

Script: `analyze_food_deep.py`

1.1 Food Production Formula

We filter to non-raided, alive, same-owner transitions ($N = 190{,}079$) and compare several linear models for $\Delta f = f_{t+1} - f_t$. The key finding is that **terrain effects scale with** $(1 - f)$, i.e. food production is dampened when food is already high. This improved R^2 from 0.51 to 0.71.

Best linear model (Model D, $R^2 = 0.714$):

$$\Delta f = \underbrace{0.031}_{\text{base}} \underbrace{-0.131 \cdot p}_{\text{pop cost}} + \underbrace{0.683 \cdot f(1-f)}_{\text{feedback}} + \underbrace{(0.050 \cdot n_P + 0.088 \cdot n_F - 0.004 \cdot n_M - 0.020 \cdot n_S)(1-f)}_{\text{terrain}} + w_{r,s,t} \tag{1}$$

where p = population, f = food, n_P, n_F, n_M, n_S = adjacent plains, forest, mountain, and settlement counts, and $w_{r,s,t}$ is a stochastic weather component constant within each (round, seed, step) group.

Table 1: Food model coefficient stability across 9 rounds.

Coefficient	Mean	Std	CV
`pop`	−0.145	0.022	0.15
`food×(1-food)`	+0.781	0.157	0.20
`plains×(1-f)`	+0.050	0.004	0.07
`forest×(1-f)`	+0.088	0.006	0.07
`mountain×(1-f)`	−0.003	0.004	1.30
`settl×(1-f)`	−0.012	0.009	0.76
`is_port`	+0.009	0.006	0.73

The terrain coefficients (plains, forest) are highly stable across rounds (CV $\leq$ 7%). The feedback term and intercept vary more, likely because they absorb round-specific parameter differences.

1.2 Weather and Winter

Using fixed-effects estimation (demeaning by round/seed/step), we extract the weather component $w_{r,s,t}$:

- Mean: +0.007, Std: 0.045, Range: $[-0.14, +0.13]$
- Within-group residual std: 0.068 (remaining settlement-level noise)
- Weather is approximately i.i.d. normal across steps, with no trend over time

The intercept of +0.031 represents food_base − winter_severity. From newborn ruin settlements (unraided, $N = 10{,}075$), which start with food ≈ 0 and receive one round of food production plus winter, we estimate child food $= 0.136 \pm 0.052$. This is consistent with winter ≈ 0.01–0.02 per step—a small effect relative to food production.

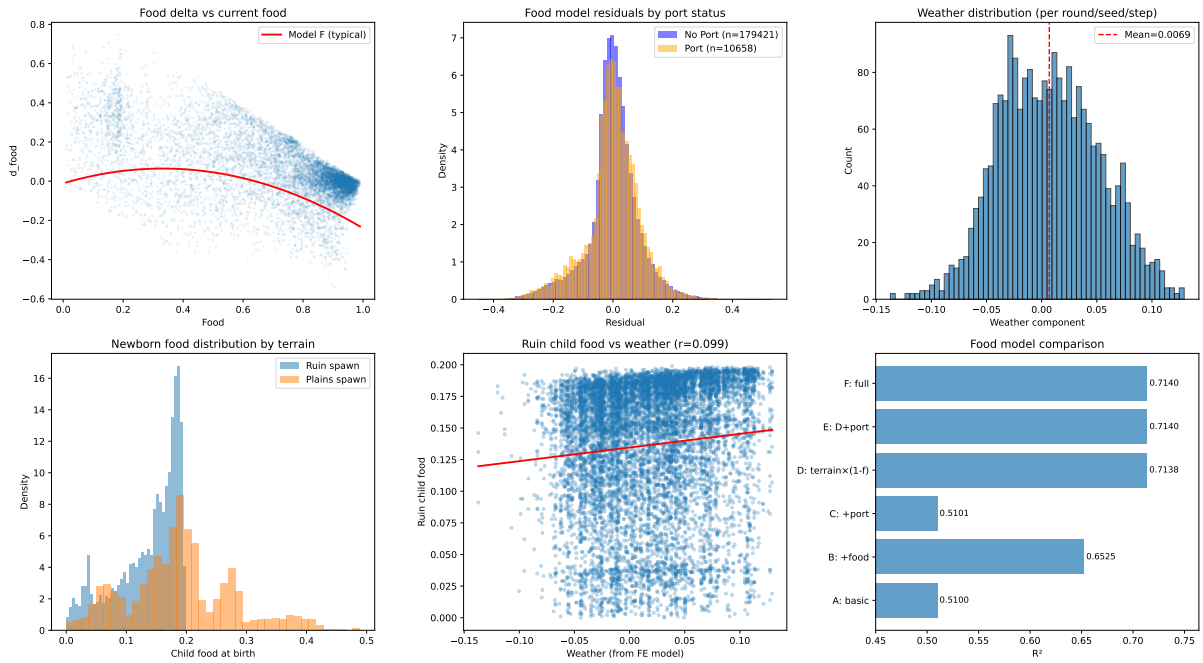

Figure 1: Food model analysis. Top-left: Δf vs f with model overlay. Top-middle: residuals by port status. Top-right: weather distribution. Bottom-left: newborn food by terrain. Bottom-middle: ruin child food vs weather ($r = 0.10$). Bottom-right: model comparison.

2 Settlement Spawning

Scripts: `analyze_spawn.py`, `analyze_spawn_pspawn.py`

2.1 Spawn Probability

From 227,719 alive settlement-steps, 22,021 resulted in a spawn (9.67%). The spawn probability is well approximated by:

$$P(\text{spawn}) \approx 0.124 \cdot p \cdot f \tag{2}$$

This is roughly linear in both population and food, though it saturates at high population ($p > 4$) and drops at very high food ($f > 0.85$). Available buildable terrain nearby (plains + forest + ruin) is a prerequisite: $P(\text{spawn})$ drops to near zero with no buildable neighbors. Defense, wealth, and port status have no direct effect.

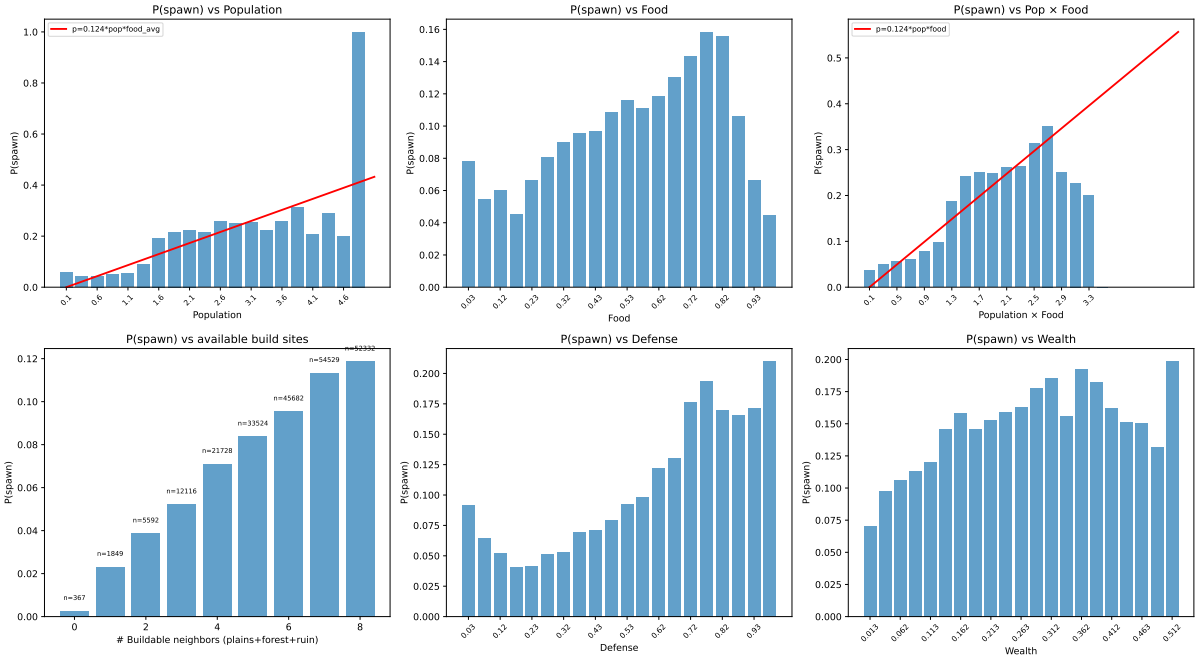

Figure 2: Spawn probability as a function of population, food, their product, buildable neighbors, defense, and wealth.

2.2 Parent and Child Effects

Parent cost. Spawning parents lose ~ 0.05 extra population compared to non-spawning settlements ($\Delta p_{\text{spawn}} = -0.063$ vs $\Delta p_{\text{no-spawn}} = -0.011$). Food and wealth are not directly affected.

Child stats. Child stats are deterministic by the terrain the child spawns on:

Table 2: Child settlement initial stats by terrain type.

Terrain	Population	Defense	Food	Wealth
Ruin ($N = 10{,}075$)	0.400 (exact)	0.150 (exact)	0.136 ± 0.052	0.007 ± 0.011
Plains ($N = 10{,}164$)	0.488 ± 0.033	0.193 ± 0.017	0.181 ± 0.088	0.031 ± 0.034
Forest ($N = 3{,}846$)	0.488 ± 0.033	0.193 ± 0.017	0.183 ± 0.086	0.031 ± 0.034

Ruin children have *exactly* pop $= 0.4$, def $= 0.15$. Plains/forest children have pop ≈ 0.5, def ≈ 0.2, with small variation from post-spawn raids and winter. Child food is weakly correlated with parent food ($r = 0.28$) and terrain neighbors.

2.3 Spawn Terrain and Distance

New settlements spawn on: Plains (42.2%), Ruin (41.8%), Forest (16.0%). Approximately 1% of coastal spawns become ports. The parent-child Chebyshev distance is: 1 (65%), 2 (26%), 3+ (9%).

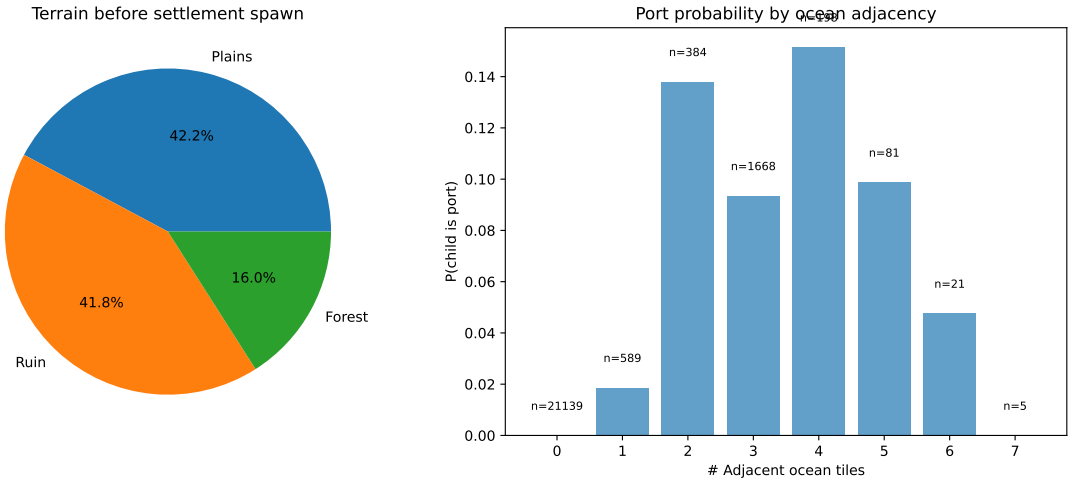

Figure 3: Left: terrain distribution before spawn. Right: port probability by ocean adjacency.

3 Raid Mechanics

Script: `analyze_raids_v2.py`

We detect raids as transitions with $\Delta\text{defense} < -0.01$, identifying 18,485 raid events (8.8% of alive settlements per step).

3.1 Defense Recovery

Non-raided settlements show defense *growth* approximately proportional to current defense:

Table 3: Defense recovery by defense level (non-raided settlements).

Defense bin	Mean Δdefense	N
0.1–0.2	+0.016	11,496
0.2–0.3	+0.023	21,298
0.4–0.5	+0.044	7,922
0.6–0.7	+0.059	5,208
0.8–0.9	+0.069	3,300
0.95–1.0	+0.002	40,541

Defense grows roughly as $\Delta d \approx 0.08 \cdot d$, capped at $d = 1.0$. A quadratic model $\Delta d = 0.349d - 0.276d^2$ achieves $R^2 = 0.59$ (non-raided, $N = 190{,}161$).

3.2 Raid Damage

Defender effects (unsuccessful raids, $N = 17{,}477$):

- Defense: $\Delta d = -0.024$ (single raid), $\Delta d = -0.094$ (2+ raids)
- Population: survival ratio $\text{pop}_{t+1}/\text{pop}_t \approx 0.91$ (single), ≈ 0.86 (heavy)
- Wealth: $\Delta w \approx -0.34 \times w$ (proportional to defender wealth)
- Food: **not directly affected** by raids ($\Delta f_{\text{raided}} \approx \Delta f_{\text{non-raided}}$)

Defender effects (successful raids / takeover, $N = 1{,}008$):

- Mean $\Delta d = -0.059$, mean $\Delta p = -0.088$, mean $\Delta w = -0.029$
- Takeover probability decreases with post-raid defense but can occur up to $d_{\text{after}} \approx 0.42$ (95th percentile)

Attacker effects cannot be cleanly isolated because the nearest enemy may not be the actual attacker, and attacker changes include natural food/pop growth. From the raw data, attackers gain small amounts of wealth (+0.004 mean).

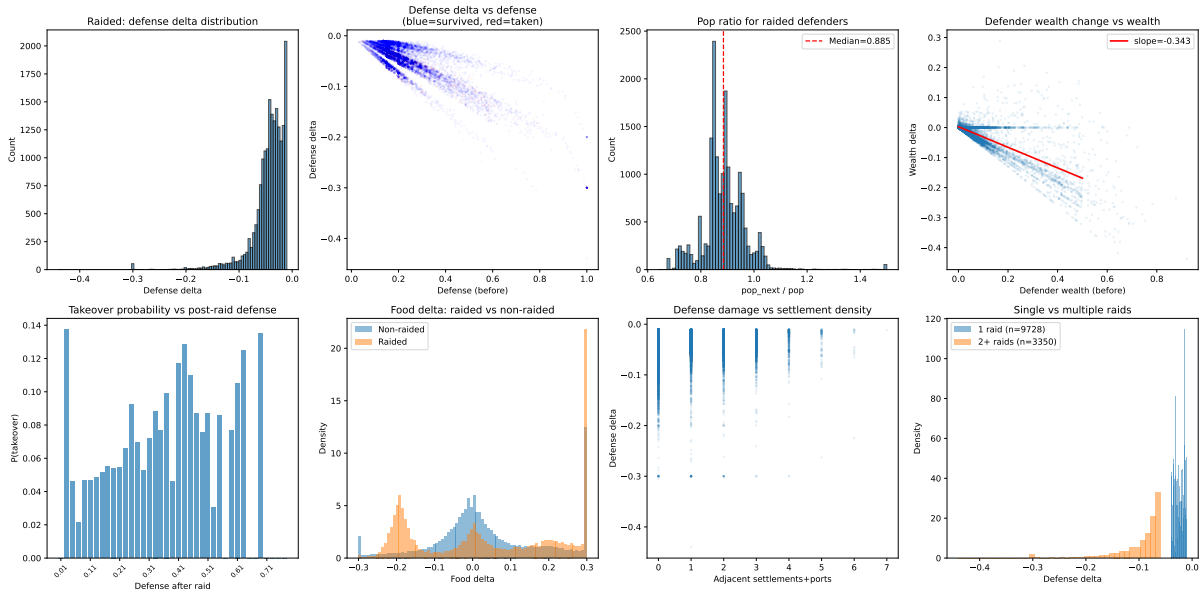

Figure 4: Raid mechanics. Top row: defense delta distribution, defense delta vs defense (blue = survived, red = taken), pop survival ratio, wealth change. Bottom row: takeover probability, food delta comparison, damage vs density, single vs multiple raids.

4 Model Quality Comparison

Script: `analyze_worst_modeled.py`

Table 4: Best linear model R^2 for each variable (non-raided, $N = 190{,}161$).

Variable	R^2 (Linear)	MAE	RMSE	R^2 (GBM CV)
Food	**0.714**	0.061	0.086	0.740
Defense	0.593	0.012	0.016	0.206
Wealth	0.021	0.005	0.015	0.074
Population	*0.011*	0.051	0.084	0.028

Population is the worst-modeled variable ($R^2 = 0.01$). Even gradient boosting achieves only $R^2 = 0.03$. The reason: population changes are dominated by stochastic events (spawning costs, raids) that are binary and unpredictable from the current state alone. The deterministic growth component ($\Delta p \approx 0.02 \cdot f \cdot (1 - p/5)$) is small relative to these shocks.

Wealth is similarly hard to predict ($R^2 = 0.02$), driven by stochastic trade and raid events.

Defense has a strong quadratic model ($R^2 = 0.59$) but GBM overfits (train $R^2 = 0.83$ vs CV $R^2 = 0.21$), indicating that non-raided defense dynamics are well captured by $\Delta d \approx 0.35d - 0.28d^2$ but additional features add noise, not signal.

Food is the best modeled ($R^2 = 0.71$), with terrain$\times(1-f)$ being the key insight. GBM only marginally improves it ($R^2 = 0.74$), suggesting the linear model captures most of the structure.

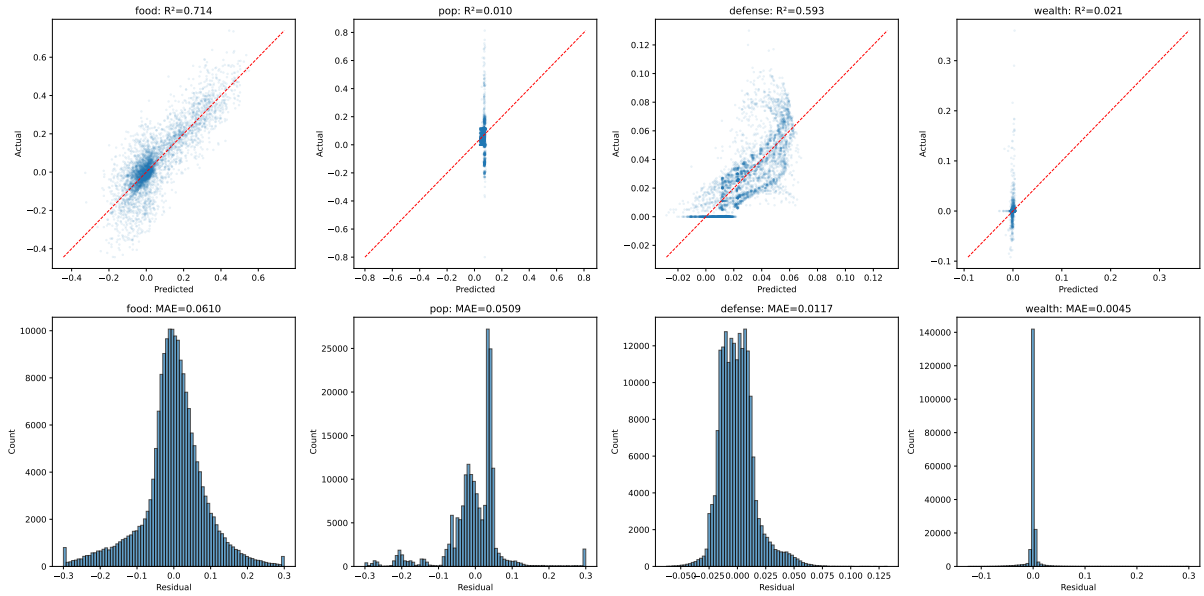

Figure 5: Model quality for all four variables. Top: predicted vs actual. Bottom: residual distributions.

The large population residuals occur disproportionately at high pop ($\bar{p} = 1.94$ vs 1.15 normal) and high defense ($\bar{d} = 0.82$ vs 0.49), consistent with spawning parents who stochastically lose population.

5 Summary of Key Parameters

Table 5: Estimated game parameters from data.

Parameter	Description	Estimated Value
`food_base - winter`	Net food intercept	$+0.031$
`food_pop_coeff`	Population food cost	-0.131
`food_feedback`	Food autoregulation $f(1-f)$	$+0.683$
`food_plains` $\times(1-f)$	Plains adjacency effect	$+0.050$
`food_forest` $\times(1-f)$	Forest adjacency effect	$+0.088$
`weather_std`	Per-step food noise	0.045
`growth_prob`	Spawn coefficient	$0.124 \times p \times f$
`parent_cost`	Pop cost of spawning	~ 0.05
`ruin_child_pop`	Ruin child population	0.400
`ruin_child_def`	Ruin child defense	0.150
`plains_child_pop`	Plains/forest child population	0.500
`plains_child_def`	Plains/forest child defense	0.200
`defense_growth`	Defense growth rate	$\sim 0.08 \cdot d$
`raid_pop_survival`	Pop retention per raid	~ 0.91
`raid_wealth_fraction`	Wealth stolen per raid	$\sim 0.34 \times w$

Reproducibility

All analysis scripts are in `data-analysis/`:

- `load_replays.py` — shared data loader (transition, spawn, raid DataFrames)
- `analyze_food_deep.py` — food model comparison, weather extraction, winter severity
- `analyze_spawn.py` — spawn statistics, parent/child effects, terrain distribution
- `analyze_spawn_pspawn.py` — spawn probability functional form, child food determinants
- `analyze_raids_v2.py` — raid detection, defense damage, pop survival, wealth, takeover
- `analyze_worst_modeled.py` — cross-variable model comparison, GBM benchmark